



Universidad autónoma de baja California

Ingeniería en computación

Sistemas embebidos

Practica 7 wifi

Evangelina Lara Camacho

Garcia Chávez Erik - 01275863

LERMA GARCIA ERICK JOVAN – 01281695

Viernes 7 de noviembre del 2025

Wifi Aware (NAN):

Es un protocolo que permite a los dispositivos WIFI descubrir servicios cercanos. utiliza Wi-Fi directo entre dispositivos para descubrir e intercambiar información. NAN escala eficazmente en entornos Wi-Fi densos y complementa la conectividad Wi-Fi al proporcionar información sobre personas y servicios cercanos.

Varios dispositivos NAN cercanos forman un clúster (**grupo de computadoras o servidores que se conectan y trabajan juntos como un único sistema para realizar tareas de manera más rápida y eficiente**) NAN pueden anunciar (método de publicación) o buscar (método de suscripción) servicios mediante los protocolos de descubrimiento de servicios NAN.

ESP32 admite Wi-Fi Aware en modo independiente, con soporte para detección de servicios y ruta de datos. Wi-Fi Aware es un protocolo en desarrollo

Para habilitar dicha utilidad es necesario configurar la estructura que guarda los parámetros de este la cual sería:

```
#include "esp_wifi.h"
#include "esp_nan.h"

void wifi_init_nan() {
    // Inicializacion basica de WiFi
    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    // Configurar modo NAN
    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_NAN));

    // Configurar parametros NAN
    wifi_nan_config_t nan_cfg = {
        .master_pref = 1, // Preferencia como maestro
        .cluster_id = 0,  // ID de clúster (0 para auto-generar)
    };
    ESP_ERROR_CHECK(esp_wifi_nan_set_config(&nan_cfg));

    // Iniciar WiFi
    ESP_ERROR_CHECK(esp_wifi_start());
}
```

Otro concepto importante dentro de WIFI Aware es **Discovery Window (DW)**

Este lo que hace es que como WIFI consume mucha energía lo que se hace es todos los dispositivos mantienen el WIFI apagado para ahorrar la energía entonces por un momento todos los dispositivos se despiertan al mismo tiempo y se empiezan a buscar mutuamente, ese momento de sincronizado es el

Discovery window. Una duración típica entre dormido y despierto puede ser de 524 mS.

El **publicar un servicio** es una funcionalidad o capacidad que ofrece el dispersivo **ESP32** a otros dispersivos cercanos

```
wifi_nan_publish_cfg_t publish_cfg = {
    .service_name = "mi_servicio",
    .type = NAN_PUBLISH_UNSOLICITED,
    .period = 1, // Publicar cada DW
};

uint8_t publish_id;
esp_wifi_nan_publish_service(&publish_cfg, &publish_id);
```

El suscribir su nombre puede ser confuso, con respecto a lo que hace, en realidad este busca quien está ofreciendo la funcionalidad o la capacidad mas no la toma.

```
wifi_nan_subscribe_cfg_t subscribe_cfg = {
    .service_name = "mi_servicio",
    .type = NAN_SUBSCRIBE_ACTIVE,
};

uint8_t subscribe_id;
esp_wifi_nan_subscribe_service(&subscribe_cfg, &subscribe_id);
```

Al momento de iniciar, de publicas o suscribirse tenemos los siguientes eventos importantes

```

static void nan_event_handler(void *arg, esp_event_base_t event_base,
                             int32_t event_id, void *event_data) {
    switch (event_id) {
        case WIFI_EVENT_NAN_STARTED:
            // NAN iniciado correctamente
            break;

        case WIFI_EVENT_NAN_STOPPED:
            // NAN detenido
            break;

        case WIFI_EVENT_NAN_SVC_MATCH:
            // Servicio encontrado
            wifi_event_nan_svc_match_t *match =
                (wifi_event_nan_svc_match_t *)event_data;
            // Usar match->publish_id, match->subscribe_id
            break;

        case WIFI_EVENT_NAN_REPLIED:
            // Recibida respuesta
            break;

        case WIFI_EVENT_NAN_RECEIVE:
            // Datos recibidos
            wifi_event_nan_receive_t *receive =
                (wifi_event_nan_receive_t *)event_data;
            break;
    }
}

```

El evento **WIFI_EVENT_NAN_STARTED** indica que el **radar** NAN esta encendido y activo.

El evento **WIFI_EVENT_NAN_STOPPED** este confirma que el módulo **NAN** se ha apagado.

El evento **WIFI_EVENT_NAN_SVC_MATCH** este es uno de los más importantes, ocurre cuando el dispositivo esta **suscrito a un servicio** y detecta a otro dispositivo que está **publicando** el mismo servicio.

El código **(wifi_event_nan_svc_match_t *)event_data** convierte los datos del evento en una estructura útil.

Esta estructura match contiene información vital, como:

- match->publish_id: El ID del servicio que *el otro dispositivo* publicó.
- match->subscribe_id: El ID de la suscripción que *tu dispositivo* creó (para que sepas cuál de tus búsquedas tuvo éxito).
- match->peer_addr: La dirección MAC del dispositivo que encontraste.

El evento **WIFI_EVENT_NAN_REPLIED** ocurre cuando un **suscriptor** te “encuentra” el **ESP** que esta de publicador. Y envía un mensaje de seguimiento. Es la forma en que el publicador se entera de que "alguien está interesado en mi servicio"

El evento **WIFI_EVENT_NAN_RECEIVE** es Cuando otro dispositivo NAN te envía un mensaje directo *sin* necesidad de establecer una conexión de datos completa

- El código (wifi_event_nan_receive_t *)event_data convierte los datos en una estructura de recepción.
- Esta estructura recibe contiene:
 - o receive->peer_addr: La dirección MAC de quién envió el mensaje.
 - o receive->data: Los datos en sí (un array de bytes).
 - o receive->data_len: El tamaño de los datos recibidos.

Casos de uso:

- Transferencia peer-to-peer
- Formar redes malladas dinámicas
- Apps que detectan usuarios cercanos
- Sensores comunicándose sin router

Desarrollo de la practica:

El problema nos indica desarrollar el juego **conecta 4** utilizando el protocolo WIFI del ESP32

Indicaciones:

- cada contador contara con 21 fichas
- si un jugador gano se muestra **felicidades jugador X**. donde X es el nombre del jugador ganador
- si ambos perdieron se muestra. **¡Suerte para la próxima!**
- si se termina el juego por medio del botón, el **jugador x salió del juego**.
Donde X es el nombre del jugador que presionó el botón

Librerías

Archivo main.c

Las librerías nuevas que se utilizaran para WIFI son:

```
#include<esp_wifi.h>
#include<esp_spiffs.h>
#include<esp_flash.h>
#include<nvs_flash.h>
#include<esp_netif.h>
#include<esp_http_server.h>
```

Estas librerías se encargarán de escribir en flash los datos de la página web, así como las conexiones que se realicen, así como traer las funciones para iniciar wifi y el servidor http local que se necesitara para mostrar la página.

Librerías propias

Estas librerías son las que tienen funciones, estructuras, colas, grupos de eventos que se necesitan para manejar tanto la lógica del juego, la lógica de establecer la conexión WIFI hacer que el ESP sea un acces point y pueda servir la página web, las funciones y aspectos relacionado con el juego, así como los grupos de eventos que

```
//librerias propias
#include"modulos/GPIO/gpio_lib.h"
#include"modulos/WIFI/wifi_lib.h"
#include"game.h"
```

Tenemos las siguientes variables globales, **game_state** es una estructura en donde se define los aspectos del juego, la medida del tablero, miembros que guardan los nombres de los usuarios, sus fichas, sus movimientos, etc.

La cola funciona solo para enviar desde el main.c a **gpio_lib.c** la cola en donde se estará mandando el botón que se presionó

Tenemos los grupos de eventos que son los que me indicaran cuando puede pasar algo y cuando no, el bit **player_name_ready_bit** indica que los nombres de los jugadores se han ingresado por lo que se puede seguir al juego, el bit **new_move_bit** indica cuando se ha movido una ficha indicando que se debe de actualizar el tablero, el bit **game_over_bit** me indica que el juego termino, porque alguien gano, se acabaron las fichas o alguien presiono el botón los bits **py1_btn_bit**, **py2_btn_bit** indican que jugador presiono el botón el bit **update_display** indica cuando se puede actualizar el tablero.

```
GameState game_state;

QueueHandle_t button_queue = NULL;

EventGroupHandle_t event_game;

const int player_name_ready_bit = BIT0, new_move_bit=BIT1, game_over_bit=BIT2, py1_btn_bit=BIT3,py2_btn_bit=BIT4, update_display=BIT5;
```

Tenemos nuestra función **app_main()** en la cual iniciamos la cola y los grupo de eventos, la función **nvs_flash_init()** trata lo que hace es tratar de iniciar el sistema para guardar datos y configuraciones en la memoria flash, la memoria no volátil, esto lo hacemos para guardar las credenciales de **WIFI** para que inicie más rápido y no tenga que configurar todo desde un inicio, una vez escrito por primera vez solo lo traer la segunda vez.

después iniciamos las demás funciones y tareas.

```
2
3 void app_main(void){
4
5
6     button_queue = xQueueCreate(10, sizeof(uint8_t));
7
8     event_group= xEventGroupCreate();
9
10    esp_err_t ret = nvs_flash_init();
11    if(ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND){
12        ESP_ERROR_CHECK(nvs_flash_erase());
13        ESP_ERROR_CHECK(nvs_flash_init());
14    }
15
16    //servidor - WIFI
17    init_spiffs();
18    wifi_init_softap();
19    start_web_server();
20
21    //funciones de juego inicales
22    //iniciamos el juego
23
24
25    //perifericos
26
27    get_queue(&button_queue);
28
29    gpio_init();
30    type_int();
31
32    //tarea de perifericos
33
34    xTaskCreate(button_task,"procces_int", 2048, NULL, 8, NULL);
35    xTaskCreate(game_task, "main_task", 4098, NULL, 10, NULL);
36    xTaskCreate(display_task, "display_task", 4096, NULL, 7, NULL);
37
38 }
39
```

La tarea **button_task** lo único que está haciendo es esperar a que un botón se presione, un GPIO este asociado a un jugador.

```
#define PLAYER_1 18 //player 1
#define PLAYER_2 19 //player 2
```

```
void button_task(void *params) {
    uint32_t pin_number;

    while (1) {
        if (xQueueReceive(button_queue, &pin_number, portMAX_DELAY)) {
            if (pin_number == PLAYER_1) {
                ESP_LOGI(TAG, "Jugador 1 finalizo el juego");
                xEventGroupSetBits(event_game, py1_btn_bit);
            } else {
                ESP_LOGI(TAG, "Jugador 2 finalizo el juego");
                xEventGroupSetBits(event_game, py2_btn_bit);
            }
        }
    }
}
```

La tarea **game_task** se encarga de llevar el control de la lógica del juego, de cuando un jugador puede ingresar su ficha, de controlar y estar checando si ya un jugador gana.

Lo primero que está haciendo es esperar que se hayan ingresado los nombres de los 2 jugadores estos bits se establecer en la función **start_player_names()** la cual se encuentra en el archivo **wifi.c**

Cuando ese bit se sete, es cuando el juego debe de empezar por lo que iniciamos los parámetros de la estructura e indicamos que puede actualizar el tablero todo estará vacío al inicio

Después debemos esperar que un jugador haga su movimiento o termine la partida. Cualquiera de esos casos dejara de estar bloqueada la tarea y debería de continuar, ahora debemos de saber cuál fue lo que se realizó.

En el caso que se eligiera el movimiento debe de checar si es que se ha ganado.

```
if (bits & new_move_bit) {
    // El movimiento ya fue procesado en el handler HTTP
    // Solo necesitamos verificar el estado del juego
    int winner = check_winner();

    if (winner) {
        // Hay un ganador
        snprintf(game_state.winner_message, sizeof(game_state.winner_message),
                 "FELICIDADES %s!",
                 (winner == 1) ? game_state.player1_name : game_state.player2_name);
        game_state.game_over = 1;
        xEventGroupSetBits(event_game, game_over_bit | update_display);
    }
}
```


La función regresa cual es el jugador que ha ganado 1 o 2. Regresa 0 si no hay ganadores.

Lo que hace es imprimir el nombre del jugador ganador.

Pero si no ha ganado nadie entonces debe de checar si se han acabado las fichas o si es turno del otro jugador de mover sus fichas,

En caso que se hayan acabado ambas las fichas y nadie gano se activa el bit que indica el fin del juego en otro caso se activa el bit que actualiza el tablero que mostrara que es el turno del siguiente jugador de hacer su jugada. Con el elemento `current_player` indicamos si es 1 o es el 2.

```
} else if (game_state.player1_chips == 0 && game_state.player2_chips == 0) {  
    // Empate  
    strcpy(game_state.winner_message, "Suerte para la próxima!");  
    game_state.game_over = 1;  
    xEventGroupSetBits(event_game, game_over_bit | update_display);  
} else {  
    // Cambiar turno  
    game_state.current_player = (game_state.current_player == 1) ? 2 : 1;  
    xEventGroupSetBits(event_game, update_display);  
}
```

En otro caso es que se presionó el botón entonces el juego debe de terminar, por lo que indicamos que jugador fue el que termino y seteamos tanto **game_over** y **update_display** para bloquear el tablero e indicar que se terminó el juego.

```
if (bits & py1_btn_bit) {  
    // Jugador 1 presiono el boton  
    snprintf(game_state.winner_message, sizeof(game_state.winner_message),  
             "El jugador %s salió del juego.", game_state.player1_name);  
    game_state.game_over = 1;  
    xEventGroupSetBits(event_game, game_over_bit | update_display);  
}  
  
if (bits & py2_btn_bit) {  
    // Jugador 2 presiono el boton  
    snprintf(game_state.winner_message, sizeof(game_state.winner_message),  
             "El jugador %s salió del juego.", game_state.player2_name);  
    game_state.game_over = 1;  
    xEventGroupSetBits(event_game, game_over_bit | update_display);  
}
```

```

void game_task(void *params) {
    // Esperar a que se ingresen los nombres de los jugadores
    xEventGroupWaitBits(event_game, player_name_rady_bit, pdTRUE, pdTRUE, portMAX_DELAY);

    // Inicializar juego
    init_game();
    xEventGroupSetBits(event_game, update_display);

    while (1) {
        // Esperar por eventos del juego
        EventBits_t bits = xEventGroupWaitBits(
            event_game,
            new_move_bit | py1_btn_bit | py2_btn_bit,
            pdTRUE, // Clear on exit
            pdFALSE, // Wait for any bit
            portMAX_DELAY
        );

        if (bits & new_move_bit) {
            // El movimiento ya fue procesado en el handler HTTP
            // Solo necesitamos verificar el estado del juego
            int winner = check_winner();

            if (winner) {
                // Hay un ganador
                snprintf(game_state.winner_message, sizeof(game_state.winner_message),
                    "FELICIDADES %s!",
                    (winner == 1) ? game_state.player1_name : game_state.player2_name);
                game_state.game_over = 1;
                xEventGroupSetBits(event_game, game_over_bit | update_display);
            } else if (game_state.player1_chips == 0 && game_state.player2_chips == 0) {
                // Empate
                strcpy(game_state.winner_message, "Suerte para la próxima!");
                game_state.game_over = 1;
                xEventGroupSetBits(event_game, game_over_bit | update_display);
            } else {
                // Cambiar turno
                game_state.current_player = (game_state.current_player == 1) ? 2 : 1;
                xEventGroupSetBits(event_game, update_display);
            }
        }

        if (bits & py1_btn_bit) {
            // Jugador 1 presionó el botón
            snprintf(game_state.winner_message, sizeof(game_state.winner_message),
                "El jugador %s salió del juego.", game_state.player1_name);
            game_state.game_over = 1;
            xEventGroupSetBits(event_game, game_over_bit | update_display);
        }

        if (bits & py2_btn_bit) {
            // Jugador 2 presionó el botón
            snprintf(game_state.winner_message, sizeof(game_state.winner_message),
                "El jugador %s salió del juego.", game_state.player2_name);
            game_state.game_over = 1;
            xEventGroupSetBits(event_game, game_over_bit | update_display);
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

La tarea **display_task** está esperando a que el bit **update display** se active para poder actualizar las fichas, por ejemplo para aquí imprime por la terminal

```
2 void display_task(void *params) {
3     while (1) {
4         // Esperar a que se necesite actualizar la pantalla
5         xEventGroupWaitBits(event_game, update_display, pdTRUE, pdTRUE, portMAX_DELAY);
6
7         ESP_LOGI(TAG, "Estado del juego actualizado");
8         ESP_LOGI(TAG, "Jugador 1: %s - Fichas: %d", game_state.player1_name, game_state.player1_chips);
9         ESP_LOGI(TAG, "Jugador 2: %s - Fichas: %d", game_state.player2_name, game_state.player2_chips);
10
11         vTaskDelay(pdMS_TO_TICKS(500));
12     }
13 }
```

Función **init_game()** se está encargando de inicializar por parámetros por defecto del juego como el iniciar el tablero con sus espacios vacíos, iniciar dándole a cada jugador sus 21 fichas, etc.

```
void init_game(void) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            game_state.board[i][j] = ' ';
        }
    }

    // strcpy(game_state.player1_name, "Jugador 1");
    // strcpy(game_state.player2_name, "Jugador 2");

    game_state.player1_chips = 21;
    game_state.player2_chips = 21;
    game_state.current_player = 1;
    game_state.game_over = 0;
    game_state.last_move_col = -1;
    game_state.last_move_row = -1;
    strcpy(game_state.winner_message, "");
}
```

Función **make_move(int column)** esta función lo que está tratando es de poner una ficha en la columna dicha por el jugador, pero que es lo que puede pasar?, que en efecto se pueda colocar la ficha en la columna especificada regresa la columna especificando que el insertar la ficha fue exitoso en el caso de que el tablero este lleno o se haya terminado el juego entonces enviar el -1, este indica que alguno de esos 2 procesos sucedieron la forma de saber cuál es que se activó algún bit.

```

int make_move(int column) {
    if (column < 0 || column >= COLS || game_state.game_over) {
        return -1;
    }

    for (int row = ROWS - 1; row >= 0; row--) {
        if (game_state.board[row][column] == ' ') {
            game_state.board[row][column] = (game_state.current_player == 1) ? 'O' : 'X';
            game_state.last_move_col = column;
            game_state.last_move_row = row;

            if (game_state.current_player == 1) {
                game_state.player1_chips--;
            } else {
                game_state.player2_chips--;
            }

            return row;
        }
    }
    return -1;
}

```

Función **check_winner()** como su nombre lo indica estará checando si cuando se movió la ficha este resultado en un ganador, en caso de ser verdadero regresa el número del jugador actual, 1 o 2, en caso contrario regresa 0 indicando que aún no hay ganadores por lo que el juego puede continuar.

Primero verifica si hay menos un movimiento.

La forma en cómo se puede ganar es conectar 4 fichas de la misma categoría en forma, vertical, horizontal o en diagonal.

Gracias al miembro de la estructura **last_move_row** y **last_move_col** no necesitamos verificar todo el tablero, solo necesitamos ver alrededor de la última ficha que se colocó si es que alrededor de esta hay 4 fichas consecutivas indicando que el jugador gana. Entonces para eso sacamos la ficha, no sabemos si es 'X' o 'O' entonces con esto lo sacamos de la matriz.

En una dirección positiva, en cada paso calculamos la nueva posición, primero vamos a verificar en forma "positiva" en horizontal. Incrementamos **count** siempre y cuando la ficha este dentro del tablero y sea de la misma categoría en otro caso rompe el ciclo, porque no se logró las 4 consecutivas lo mismo aplica hacia los otros sentidos en forma horizontal hacia la izquierda y de arriba-izquierda y abajo-derecha.

```

int check_winner(void) {
    if (game_state.last_move_row == -1 || game_state.last_move_col == -1) {
        return 0;
    }

    char current_piece = game_state.board[game_state.last_move_row][game_state.last_move_col];
    int directions[4][2] = {{1,0}, {0,1}, {1,1}, {1,-1}};

    for (int d = 0; d < 4; d++) {
        int count = 1;

        //direccion positiva
        for (int i = 1; i < 4; i++) {
            int new_row = game_state.last_move_row + i * directions[d][0];
            int new_col = game_state.last_move_col + i * directions[d][1];

            if (new_row >= 0 && new_row < ROWS && new_col >= 0 && new_col < COLS &&
                game_state.board[new_row][new_col] == current_piece) {
                count++;
            } else {
                break;
            }
        }

        //direccion negativa
        for (int i = 1; i < 4; i++) {
            int new_row = game_state.last_move_row - i * directions[d][0];
            int new_col = game_state.last_move_col - i * directions[d][1];

            if (new_row >= 0 && new_row < ROWS && new_col >= 0 && new_col < COLS &&
                game_state.board[new_row][new_col] == current_piece) {
                count++;
            } else {
                break;
            }
        }

        if (count >= 4) {
            return game_state.current_player;
        }
    }

    return 0;
}

```

Function **char* get_board_json()** el propósito de esta función es el convertir el tablero del juego que es una matriz de carácter en una cadena JSON esto para enviar el estado del tablero a través de la web por el protocolo HTTP

El arreglo **json** es un arreglo estático que almacenara el JSON resultante. Va a iterar sobre cada fila del tablero agrupándolos todos dentro del arreglo principal. Después sobre cada columna, en este momento ya tiene él lo que hay en cada espacio del tablero por lo que lo toma y lo guarda en la variable **piece**, usando **sprintf** formatea ese carácter como una cadena JSON y lo guarda en el arreglo **temp** usando la función **strcat** concatena la cadena **temp** al string **json**. Se le agrega comillas entre los elementos y entre los arreglos exceptos los últimos tanto de caracteres como el ultimo arreglo

```

char* get_board_json(void) {
    static char json[1024];
    char temp[100];

    strcpy(json, "[");
    for (int row = 0; row < ROWS; row++) {
        strcat(json, "[");
        for (int col = 0; col < COLS; col++) {
            char piece = game_state.board[row][col];
            sprintf(temp, "\"%c\"", piece);
            strcat(json, temp);
            if (col < COLS - 1) strcat(json, ",");
        }
        strcat(json, "]");
        if (row < ROWS - 1) strcat(json, ",");
    }
    strcat(json, "]");

    return json;
}

```

Función **char* get_game_info_json()**

El propósito de esta función es tomar los datos actuales del juego y los formateo en una cadena JSON para ser enviada al cliente web

```

char* get_game_info_json(void) {
    static char json[512];

    sprintf(json,
        "{\"player1_name\": \"%s\", \"player2_name\": \"%s\", \"",
        "\"player1_chips\": %d, \"player2_chips\": %d, \"",
        "\"current_player\": %d, \"game_over\": %d, \"",
        "\"winner_message\": \"%s\"}",
        game_state.player1_name, game_state.player2_name,
        game_state.player1_chips, game_state.player2_chips,
        game_state.current_player, game_state.game_over,
        game_state.winner_message);

    return json;
}

```

Archive game.h

Tiene la declaración de las funciones que se utilizarán en main.c así como las estructuras, macros, grupo de eventos, colas que también utiliza **main.c** y **wifi.c** para poder la lógica entre el juego, como se recibe los datos desde el “front” hacia C y guardar los datos en los miembros de las estructuras como el nombre de los usuarios, sus movimientos y las fichas, etc.

En la estructura tenemos lo que presenta el juego, un miembro que define y tiene el estado del juego actual, los nombres de los jugadores, sus fichas que tiene, cual es el jugador actual para llevar un control de a quien le toca, 2 miembros importantes son **last_move_col** y **last_move_row** estos se agregan para la optimización del juego, sin estas se tendría que checar todo el tablero después de cada movimiento con esas solo revisamos alrededor de la **última ficha colocada** optimizando mucho más el juego.

```
//macros
#define ROWS 6
#define COLS 7

//variables
typedef struct {
    char board[ROWS][COLS]; // 'O' para jugador 1, 'X' para jugador 2, ' ' para vacío
    char player1_name[32];
    char player2_name[32];
    int player1_chips; //las fichas de que tiene cada jugador
    int player2_chips;
    int current_player; // 1 o 2 -> para controlar que jugador puede poner
    int game_over; //termina el juego por ganar o por que se preiosno el boton
    // int active_game;
    char winner_message[128]; //mensaje del ganaor en dado caso
    int last_move_col;
    int last_move_row;
}GameState;
```

Archivo Wifi.c

Dentro del archivo **wifi.c** tenemos la función **init_spiffs()** lo que está tratando de hacer es utilizar la memoria flash como si fuera un pequeño disco duro

Partes importantes de la estructura, seria **base_path** que es el punto de montaje seria como la raíz en donde se montaran las demás páginas, **max_files** el número máximo de archivos que se pueden abrir simultáneamente, entonces a partir de **spiffs** es donde estarán nuestras páginas y archivos css, js.

```
nit_spiffs(void){
    esp_vfs_spiffs_conf_t conf ={
        .base_path = "/spiffs",
        .partition_label = NULL,
        .max_files = 5,
        .format_if_mount_failed = true
    };

    esp_err_t ret = esp_vfs_spiffs_register(&conf);

    if(ret != ESP_OK){
        ESP_LOGE(TAG, "error al iniciar SPIFFS (%s)", esp_err_to_name(ret));
        return;
    }

    size_t total = 0, used = 0;

    ret = esp_spiffs_info(NULL, &total, &used);
    if(ret != ESP_OK){
        ESP_LOGE(TAG, "No se puede obtener la informacion de la particion SPIFFS (%s)", esp_err_to_name(ret));
    }
    else{
        ESP_LOGI(TAG, "SPIFFS: tamaño total %d, usado %d", total, used);
    }
}
```

La función **wifi_init_softap()** está tratando de convertir el ESP32 en un access point para que otros dispositivos se puedan conectar a él, primer está indicando la interfaz de red. Se crea un loop que maneja los eventos de manera por defecto, se podría personalizar, pero por el momento se deja así, después se crea la red específica para el modo access point.

La estructura **wifi_config_t** es una de las partes más importantes dado que aquí es en donde se recibirá la información para poder ser un access point como el nombre de la red, la contraseña de esta, la cantidad máxima de dispositivos que se pueden conectar y el tipo de seguridad que tendrá esta conexión

Una vez configurado, se pone el ESP en modo access point, aplicamos la configuración y se enciende WIFI para empezar a transmitir la red.


```

void wifi_init_softap(void){
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    esp_netif_create_default_wifi_ap();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    const char mi_ssid[] = "mi_esp_ap";

    wifi_config_t wifi_config={
        .ap ={
            .ssid_len = strlen(mi_ssid),
            .channel = 1,
            .password = "123456789",
            .max_connection = 4,
            .authmode = WIFI_AUTH_WPA_WPA2_PSK
        },
    };

    memcpy(wifi_config.ap.ssid, mi_ssid, strlen(mi_ssid));

    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
    ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &wifi_config));
    ESP_ERROR_CHECK(esp_wifi_start());

    ESP_LOGI(TAG, "ESP32 AP iniciado SSID: %s password: %s channel: %d",
             wifi_config.ap.ssid, wifi_config.ap.password, wifi_config.ap.channel);
}

```

La función `start_web_server()` esta función se encarga de iniciar el servidor web en el ESP32. Configura el servidor HTTP y registra los diferentes manejadores para las rutas que el servidor debe atender como el mostrar y cachar los datos a y de los usuarios.

Tenemos 5 manejadores registrados, esta

- 1- **root_uri** él tiene el propósito de servir la página en donde los jugadores se van a registrar. Esta muestra el archivo **index.html**
- 2- **game_uri** cuando se registran tiene el propósito de servir la página en donde se muestra el juego y todo lo que tiene que ver con el
- 3- **start_uri** recibe el nombre de los jugadores lo guarda en la estructura y redirigirá a la página del juego
- 4- **recv_post** recibe por el movimiento del jugador lo procesa y retorna el resultado
- 5- **state_usri** devuelve el estado del juego en formato JSON
- 6- **css_uri** se encarga de cargar el diseño web

```

void start_web_server(void){

    httpd_handle_t server = NULL;
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();

    //este metpp uri es la direccion raiz que se muestra, es metodo GET
    //aqui me muestra a pagina donde se registran los jugadores
    if(httpd_start(&server,&config) == ESP_OK){

        //archivo estatico
        httpd_uri_t root_uri={
            .uri="/",
            .method = HTTP_GET,
            .handler = root_get_handler,
        };

        //regitamos la uri que respondera a la peticion
        httpd_register_uri_handler(server, &root_uri);

        httpd_uri_t game_uri = {
            .uri = "/game.html",
            .method = HTTP_GET,
            .handler = game_get_handler
        };
        httpd_register_uri_handler(server, &game_uri);

        //manejador que va a registrar los usuarios
        httpd_uri_t start_uri={
            .uri="/start",
            .method=HTTP_POST,
            .handler = start_player,
        };

        httpd_register_uri_handler(server, &start_uri);

        httpd_uri_t recv_post={
            .uri="/move",
            .method=HTTP_POST,
            .handler = movent_handler,
        };

        httpd_register_uri_handler(server, &recv_post);

        httpd_uri_t state_uri = {
            .uri = "/game_state",
            .method = HTTP_GET,
            .handler = game_state_handler
        };
        httpd_register_uri_handler(server, &state_uri);

        //manejador CSS
        httpd_uri_t css_uri ={
            .uri="/styles.css",
            .method=HTTP_GET,
            .handler= css_get_handler,
            .user_ctx =NULL
        };
        httpd_register_uri_handler(server, &css_uri);

    }

}

```

Los manejadores **root_get_handler**, **game_get_handler**, **css_get_handler** son casi lo mismo ambos abren el archivo HTML del sistema SPIFFS e lo envía al cliente. Donde root abre el index mostrando la página de registro y **game** abre **game.html** que es donde está el UI del juego, y el manejador de CSS carga los estilos que le darán un poco de vida al juego

Manejaro root_get_handler

La manera de abrir el archivo es mediante las funciones normales de C para abrir archivos en el caso de no poder abrir el archivo se notifica que no se pudo abrir en otro caso, lo que se hace es enviar trozo a trozos de la página para que no resulte muy pesado para el ESP hasta que toda la página ha sido enviada. Algo similar pasa con los otros manejadores con método GET ya que solo sirven.

```
esp_err_t root_get_handler(httpd_req_t *req){

    FILE *file = fopen("/spiffs/index.html", "r");
    if(file==NULL){
        ESP_LOGE(TAG,"no se pudo abrir el archivo index.html");
        httpd_resp_send_404(req);
        return ESP_FAIL;
    }

    char line[255];
    while (fgets(line, sizeof(line), file)){

        httpd_resp_sendstr_chunk(req, line);
    }

    //cerramos la comunicacion

    httpd_resp_sendstr_chunk(req, NULL);
    fclose(file);
    return ESP_OK;
}
```

```

esp_err_t game_get_handler(httpd_req_t *req){

    FILE *file = fopen("/spiffs/game.html", "r");
    if(file==NULL){
        ESP_LOGE(TAG,"no se pudo abrir el archivo game.html");
        httpd_resp_send_404(req);
        return ESP_FAIL;
    }

    char line[255];
    while (fgets(line, sizeof(line), file)){
        httpd_resp_sendstr_chunk(req, line);
    }

    //cerramos la comunicacion

    httpd_resp_sendstr_chunk(req, NULL);
    fclose(file);
    return ESP_OK;
}

```

la diferencia con CSS es que se especifica que es un archivo CSS ya que por defecto el archivo que se espera en HTML

```

esp_err_t css_get_handler(httpd_req_t *req){

    FILE *file = fopen("/spiffs/styles.css", "r");
    if(file==NULL){
        ESP_LOGE(TAG,"no se pudo abrir el archivo para lectura");
        httpd_resp_send_404(req);
        return ESP_FAIL;
    }

    httpd_resp_set_type(req, "text/css");

    char line[255];
    while (fgets(line, sizeof(line), file)){

        httpd_resp_sendstr_chunk(req, line);
    }

    httpd_resp_sendstr_chunk(req, NULL);
    fclose(file);
    return ESP_OK;
}

```

Manejador **start_player**

Cunado en el formulario se le da al botón, el formulario envía por el método **POST** la información que se obtuvo en el “front” por medio de la función **httpd_req_resv()** indicamos el buffer en donde se va a guardar, la solicitud HTTP entrante que contiene la información del navegador, hacemos uso de una macro la que se encarga de que solo entren los valores necesarios que no exceda el tamaño del buffer esto para que no hayan errores de sobre flujo

Si la función recibe algo al final de lo que se recibió agregamos el carácter nulo para indicar el final de la entrada de datos,

Hacemos uso de la función **strstr** esto para encontrar la primera coincidencia de una subcadena dentro de nuestro buffer, dado que la respuesta recibida, en este caso seria “**player1=xxxxx&player2=xxxxx**” lo que se hace es separar las subcadenas que contengan el string dado para poder encontrar el nombre danto del jugador 1 como del jugador 2., una vez encontrado lo que se hace es recorrer el apuntador 8 posiciones que es la medida que tiene el “label” del nombre del usuario, después de esos todo lo que este a la derecha corresponde al nombre del usuario pero aún tiene caracteres que son propios de cómo se recibe la respuesta como lo es ‘&’ lo que se hace con la función **strchr()** buscamos la primera ocurrencia y se sustituye con el carácter nulo indicando el final del nombre del usuario, después se manda a la función **set_player_names()** en donde se ingresan a la estructura.

Activamos el bit indicando que se han ingresado el nombre de los usuarios el juego puede comenzar.

```
void set_player_names(const char *player1, const char *player2){  
  
    strncpy(game_state.player1_name, player1, sizeof(game_state.player1_name)-1);  
    game_state.player1_name[sizeof(game_state.player1_name)-1]='\0'; //termino la cadena  
  
    strncpy(game_state.player2_name, player2, sizeof(game_state.player2_name)-1);  
    game_state.player2_name[sizeof(game_state.player2_name)-1]='\0'; //termino la cadena  
  
    ESP_LOGI(TAG, "Nombres establecidos: %s, %s", player1, player2);  
  
}
```

```

esp_err_t start_player(httpd_req_t *req){

    char buff[32];

    //recibidneod los datos
    int ret, reaming = req->content_len;

    ret= httpd_req_recv(req,buff,MIN(reaming, sizeof(buff)));

    if(ret <=0){
        return ESP_FAIL;
    }

    //terminamos el string
    buff[ret]='\0';

    ESP_LOGI(TAG,"informacion ingresada: %s", buff);

    char *player1 = strstr(buff, "player1=");
    char *player2 = strstr(buff, "player2=");

    //es que player tiene algo, algo ingreso el ueusaior correctamsnte,
    if(player1 && player2){
        //se brinca la parte de player de abmos para ahora si estar en el nombre como
        //tal que ingreso el usuario, y sabemos que el final de esa cadena esta en terminacion
        //de caracter nulo
        player1+=8;
        player2+=8;

        char *end1 = strchr(player1, '&');
        if(end1){
            *end1 = '\0';
        }

        char *end2 = strchr(player2, '&');
        if(end2){
            *end2 = '\0';
        }

        set_player_names(player1,player2);
        xEventGroupSetBits(event_game, player_name_rady_bit);

        //redirigimos a la pantalla del juego

        httpd_resp_set_status(req, "303 See Other");
        httpd_resp_set_hdr(req, "Location", "/game.html");
        httpd_resp_send(req, NULL, 0);

        ESP_LOGI(TAG, "jugadores registrados : %s - %s", game_state.player1_name, game_state.player2_name);
    }
    else{
        httpd_resp_sendstr(req, "Error datos de envio");
        return ESP_FAIL;
    }

    return ESP_OK;
}

```

Handler movent_handler():

Este manejar recibirá una columna del 1 – 7

Antes de procesar lo que se recibió verifica si el juego termino, por lo que regresa porque no se puede ingresar más nada. En caso contrarios

Lo mismo se aplica para recibir los datos como los nombres

Limpiamos la entrada para quedarlos con el número que se ingresó.

Avanzamos 4 posiciones que es el tamaño del "label" del dato recibido es cuando usamos **atoi** para convertir el string a un número entero -1 dado que para comodidad del usuario es más familiar ver 1- 7 que 0- 6 que en programación es el índice real de las columnas

Después de ser recibida la respuesta verifica que la columna este dentro del rango, intentara colocar la ficha si no se pudo internet manda a pantalla que el movimiento no es válido, pero si lo es se ha actualiza el tablero activando el bit de **new_move_bit** y puede seguir el juego

```

esp_err_t movent_handler(httpd_req_t *req) {
    if (game_state.game_over) {
        httpd_resp_sendstr(req, "JUEGO_TERMINADO");
        return ESP_OK;
    }

    char buff[32];
    int ret, remaining = req->content_len;

    ret = httpd_req_recv(req, buff, MIN(remaining, sizeof(buff)));
    if (ret <= 0) {
        return ESP_FAIL;
    }

    buff[ret] = '\0';
    ESP_LOGI(TAG, "Movimiento recibido: %s", buff);

    char *col_str = strstr(buff, "col=");
    if (col_str) {
        col_str += 4;
        int column = atoi(col_str) - 1;

        if (column >= 0 && column < COLS) {
            int result = make_move(column);
            if (result != -1) {
                xEventGroupSetBits(event_game, new_move_bit);
                httpd_resp_sendstr(req, "OK");
            } else {
                httpd_resp_sendstr(req, "INVALID_MOVE");
            }
        } else {
            httpd_resp_sendstr(req, "INVALID_COLUMN");
        }
    } else {
        httpd_resp_sendstr(req, "INVALID_FORMAT");
    }

    return ESP_OK;
}

```


Manejador game_state_handler

Se encarga de enviar el estado actual del juego en formato JSON al cliente.

La función **snprintf** formatea una cadena json que contiene dos objetos anidados que son el **tablero get_board_json** y la información **get_game_info**

La función **httpd_resp_set_type** se indica que la después es de tipo JSON

```
// Handler para estado del juego (API JSON)
esp_err_t game_state_handler(httpd_req_t *req) {
    char response[2048];

    snprintf(response, sizeof(response),
        "{\"board\":%s,\"info\":%s}",
        get_board_json(),
        get_game_info_json());

    httpd_resp_set_type(req, "application/json");
    httpd_resp_send(req, response, strlen(response));

    return ESP_OK;
}
```

Tenemos el HTML

En especial tenemos el código JS

La función **updateGame()** realiza una petición GET a /game_state para obtener el estado actual del juego en formato JSON si el juego ha terminado esconde los controles

```
function updateGame() {
    fetch('/game_state')
        .then(response => response.json())
        .then(data => {
            updateBoard(data.board);
            updateGameInfo(data.info);

            if (data.info.game_over) {
                document.getElementById('controls').style.display = 'none';
            }
        });
}
```

Función **updateBoard(board)** recibe el tablero y genera una tabla HTML cada celda tiene una ficha O o X de caso contrario recibe un espacio en blanco

```
function updateBoard(board) {
  const gameBoard = document.getElementById('gameBoard');
  let html = '<table border="1">';

  for (let row = 0; row < board.length; row++) {
    html += '<tr>';
    for (let col = 0; col < board[row].length; col++) {
      const piece = board[row][col];
      html += `<td>${piece !== ' ' ? piece : '&nbsp;'}</td>`;
    }
    html += '</tr>';
  }

  html += '</table>';
  gameBoard.innerHTML = html;
}
```

La función **updateGameInfo(info)** está mostrando y actualizando la información de los jugadores como el número de fichas que tiene cada uno y el turno de cada quien

```
function updateGameInfo(info) {
  const currentPlayerEl = document.getElementById('currentPlayer');
  const winnerMessageEl = document.getElementById('winnerMessage');

  document.getElementById('player1Name').textContent = info.player1_name;
  document.getElementById('player2Name').textContent = info.player2_name;
  document.getElementById('player1Chips').textContent = info.player1_chips;
  document.getElementById('player2Chips').textContent = info.player2_chips;

  if (info.game_over) {
    currentPlayerEl.textContent = '';
    winnerMessageEl.textContent = info.winner_message;
  } else {
    const currentPlayer = info.current_player === 1 ? info.player1_name : info.player2_name;
    currentPlayerEl.textContent = `Turno de: ${currentPlayer}`;
    winnerMessageEl.textContent = '';
  }
}
```

La función **makeMove()** toma el valor de la columna ingresada en vía una petición POST a /move con la columna si el movimiento es válido, actualiza el juego después de 100 ms si no muestra el mensaje de error:

```
function makeMove() {
    const column = document.getElementById('column').value;

    fetch('/move', {
        method: 'POST',
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: 'col=' + column
    })
    .then(response => response.text())
    .then(result => {
        if (result === 'OK') {
            setTimeout(updateGame, 100);
            document.getElementById('column').value = '';
        } else {
            alert('Movimiento inválido: ' + result);
        }
    });
}
```

Conclusiones

Erik garcia chavez:

Se logro desarrollar un servidor que ofrece el juego de conecta 4 utilizando el ESP como acces point, se logro entender y desarrollar la meta de la practica la cual es el implementar un sistema IoT el poderse conectar con otros dispositivos mediante el protocolo WIFI

Lerma garcia erick jovan:

Se logo implementar el desarrollo utilizando diversas tecnologías como el protocolo WIFI para convertir el ESP en un acces point, el cual ofrece un servidor web embebido para la interfaz de usuario, GPIO para la interacción física y algoritmos en C para la lógica del juego.

Dificultades

Erik garcia chavez:

Una de las mayores dificultades que tuve fue el entender como es que se comunica la parte del backend escrito en C y la lógica hacia el HTML para mostrarle al usuario. El proceso de serializar las estructuras de datos del juego en formato JSON y luego parsear los datos enviados desde los formularios HTML mediante POST requirió numerosas iteraciones y pruebas. La falta de experiencia previa en JavaScript nos obligó a investigar y aprender conjuntamente cómo manejar las peticiones asíncronas y la actualización dinámica del DOM.

Lerma garcia erick jovan:

Coordinar el estado del juego entre las diferentes tareas de FreeRTOS, las interrupciones GPIO y las peticiones HTTP representó un desafío significativo.

Tuvimos que diseñar cuidadosamente el sistema de Event Groups para asegurar que los movimientos de los jugadores, las interrupciones de botones y las actualizaciones del display se sincronizaran correctamente sin condiciones de carrera o bloqueos.

Bibliografía:

- https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_server.html
- https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_wifi.html
- <https://freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/00-Tasks-and-co-routines>